Fatima *et al. Journal of Big Data* (2026) 13:77
https://doi.org/10.1186/s40537-026-01414-8

Journal of Big Data

RESEARCH Open Access

A novel stacking-based ensemble machine learning model for accurate user story effort estimation in scrum

Iman Fatima[1], Maria Rasheed[1], Darain Fatima[1], Muhammad Hamid[1*], Ines Hilali Jaghdam[2] and Ammar T. Zahary[3,4*]

*Correspondence:
Muhammad Hamid
mhamid@gcwus.edu.pk
Ammar T. Zahary
zahary@su.edu.ye
[1]Department of Computer science, Government College Women University, Sialkot 51310, Pakistan
[2]Department of Computer Science and Information Technology, Applied College, Princess Nourah bint Abdulrahman University, P.O. Box 84428, Riyadh 11671, Saudi Arabia
[3]Department of IT, Sana'a University, Sana'a, YE, Yemen
[4]Faculty of Computing and IT, University of Science and Technology, Sana'a, YE, Yemen

Abstract

Effort estimation remains a persistent challenge in agile software development, particularly at the user story level where iterative delivery is emphasized. Inaccurate estimations can result in misallocated resources, budget overruns, and project delays. This research introduces a novel stacking-based ensemble Machine Learning (ML) model designed to enhance the accuracy of user story effort estimation. To address the scarcity of granular data, a high-fidelity dataset comprising 160 user stories from 36 professional scrum-based projects was developed, incorporating 13 industry-validated effort drivers. Adopting the Design Science Research (DSR) methodology, a two-tier hierarchical ensemble model was implemented. In this architecture, Extra Trees, XGBoost, and Random Forest function as Tier-1 base learners, while Linear Regression is employed as the Tier-2 meta-learner to synthesize predictive outputs and optimize the final estimate. Furthermore, a functional web-based prototype interface was developed, operationalizing the model for real-time decision support. Model's performance was assessed using standard regression metrics: Mean Absolute Error (MAE), Mean Squared Error (MSE), and Root Mean Squared Error (RMSE). The proposed stacking ensemble model outperformed all standalone individual models, achieving MAE = 0.51, MSE = 0.49, and RMSE = 0.70. These results demonstrate the model's effectiveness in improving estimation precision, thereby facilitating superior sprint planning and resource optimization within Scrum teams. Future research will explore the integration of unstructured textual data through Natural Language Processing (NLP) techniques and the adoption of deep learning architectures to further expand the model's predictive capabilities.

Keywords Ensemble model, Machine learning, Scrum, User story

Introduction

Accurate effort estimation has been a persistent challenge in agile software development, especially in the scrum methodology where work is planned, prioritized, and delivered in the form of increments. Unlike other traditional project management methods where estimates are typically performed at project level, scrum requires the estimation to be performed at the granularity of the individual user stories to support effective sprint

© The Author(s) 2026. **Open Access** This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

planning, allocate resources efficiently and predictable delivery schedule [1, 2]. However, inaccurate estimates remain persistent problem, which causes project delays, budget overruns and lack of confidence among the team members [3]. Although the estimation of effort is influenced by numerous context-dependent factors referred to effort drivers and identifying and quantifying these effort drivers is a complicated process [4]. There are three common methods for scrum effort estimation [5]:

1. Expert-based estimation, which relies on team members' judgment.
2. Data-driven estimation, which applies ML or statistical models to historical data.
3. Hybrid estimation, combining expert judgment with empirical methods.

Although there is a transition towards the use of data-driven methods to enhance objectivity, there is still a research gap in the literature. First, the vast majority of existing models are trained on the legacy, project-level data e.g., COCOMO or NASA93, which do not offer the granularity needed to estimate the effort of user stories at the sprint level [4, 6, 7]. Second, there is lack of high-quality, industry-tested data, which considers complex real-world effort drivers [8–10]. Lastly, the existing individual ML models are likely to be poorly generalized and have high error rate when applied to the non-linear, high-dimensional dynamics of agile projects [11–13]. Moreover, most studies focus on small and novice teams and do not have the architectural complexity that is required to reflect the complexity of professional and large-scale software development setting [14–16].

This study fills these gaps by following a DSR methodology to develop a strong estimation model. The study develop stacking-based ensemble ML model with two-tier hierarchical structure. We use three high-performing regressors Extra Trees, XGBoost, and Random Forest in Tier-1 (Base Learners) to learn various patterns using randomization and gradient boosting. Their predictive outputs are further incorporated in Tier-2, where Linear Regression model acts as the meta-learner. Through this stacking architecture, the meta-learner is trained to weight the outputs of the base models in an optimal way such that the bias and variance of base models can be reduced [17, 18]. In order to confirm this methodology, we created a novel and dedicated dataset of 160 user stories derived out of 36 professional Scrum projects, with 13 granular effort drivers that were determined through systematic literature review and direct industry cooperation.

To guide this study, the following research questions are formulated:

RQ1: What factors affect accuracy in Scrum effort estimation?
RQ2: Does estimation accuracy in Scrum-based projects improve through the application of ML techniques?

The primary contributions of this research are as follows:

1. Development of a high fidelity dataset of 160 user stories from scrum development environment. The dataset is considered to be unique due to the inclusion of 13 granular features, filling the gap between the theoretical studies and the real-life agile complexity.
2. Eight different ML algorithms such as extra tree, XGBoost, random forest and linear regression, MLP, KNN, decision tree and SVR were systematically assessed. This

intensive testing is the basis of determining the strongest predictors and comparing their respective effectiveness with Scrum-specific features.

3. Design and development of novel, two tier stacking-based ensemble model. In this hierarchical model, the three most effective regressors extra tree, XGBoost and random forest are used as Base learners at Tier-1, and linear regression is used as Tier-2 meta-learner. This is a stacking strategy that maximizes accuracy of the final estimate by learning to produce the various predictive patterns of the underlying models, which reduces bias and variance.
4. Creation of an efficient web-based decision support tool. This contribution transforms the theoretical study into a practical object, allowing Scrum Masters to make the real-time, data-driven estimation of the effort during the planning of the sprint.

The paper is structured as follows: Sect. "Background and related work" presents the background and related work; Sect. "Methodology" discusses the methodology; Sect. "Results" analyzes the results; and Sect. "Conclusion and future work" concludes with the findings and future research directions.

Background and related work

Scrum is an agile methodology used for development of complex product using incremental and iterative approaches shown in Fig. 1 [5]. Instead of organizing work over a long period of time it breaks down development process into short, time-bound sprints that usually span 2–4 weeks. Every sprint produces a product increment that is ready for deployment. Scrum promotes the adaptability, open communication, close teamwork and continuous feedback to align with both customer expectations and the organization's goals. Its lightweight and flexible structure makes it effective for dynamic environments where requirements frequently evolve. There are three roles in scrum that support for carrying out successful sprint execution [1].

a) Product Owner establishes and sets priorities for the product backlog, so team members work on the most important activities for customers.
b) Scrum Master makes scrum processes easier, addresses any difficulties and helps the team apply agile principles.
c) Development team is a cross functional team that ensures product deliverables are ready with every sprint.

Scrum artifacts provide transparency and serve as key tools for planning, tracking and delivering work [1]. There are three artifacts in scrum.

a) Product Backlog: A constantly updated list of mandatory features and enhancements, arranged in order of importance.

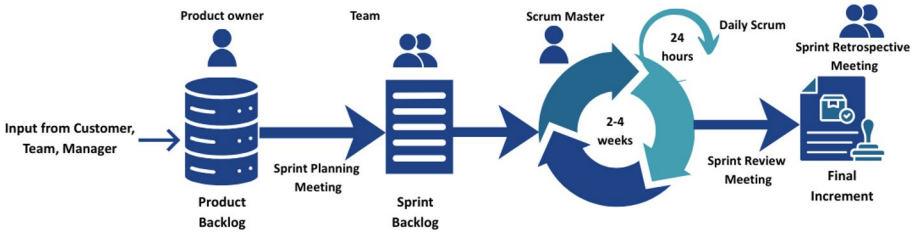


Fig. 1 Scrum methodology

b) Sprint Backlog: A subset of product backlog items that are to be finished during the current sprint.
c) Increment: The amount of backlog items completed during the sprint that can be used by customers.

The Scrum framework uses five meetings to help the team with planning, implementation and enhancements [1].

a) Sprint Planning: Makes sure everyone is clear about the sprint goal and chooses work from backlog to include in the upcoming version.
b) Daily Scrum: Having a short daily meeting among team members to exchange updates and recognize difficulties they have.
c) Sprint Review: The sprint review takes place near the end to exhibit the increment and gather the team's points of view.
d) Sprint Retrospective: Assesses the sprint and suggests what could be changed.
e) Backlog Refinement: Continual effort to sort and clarify the backlog list before starting a new sprint.

Literature review

Accurate effort estimation is the foundations of effective project management, but it is a consistent challenge, especially in the agile methodologies like Scrum [1, 8, 19]. Inaccurate estimates may cause misallocation of resources, budget overrun and missed deadlines, which compromise the nature of agility that these methodologies are meant to offer [11, 12, 20]. Over the last few years, there has been a growing trend in research towards the use of data driven methods, particularly ML, to enhance the classical methods of estimation, such as Planning Poker [21]. Another major barrier to the development of data-driven estimation of effort is the absence of publicly available, high quality and contextually specific datasets [22, 23]. Most of the research uses historical or project-level data, such as COCOMO or NASA93 [6, 7, 24], which cannot be applicable to the granular, user-story-level estimation that scrum demands.

As an example, Karna et al. [25] created their model based on the data of individual large project, which can limit the generalizability of their results. On the same note, Carvalho et al. [8] and Arora et al. [9] observed that their datasets were small, which was a limitation. This is usually a two-fold challenge because the companies do not want to disclose proprietary project data due to confidentiality reasons and publicly accessible data of open-source projects can be noisy or inconsistent [26]. This common problem was considered by Atoum et al. [27], who developed a technique to operate with small datasets. As an example, Ersoy et al. [28] conducted a large-scale case study in a FinTech company, gathering 12 months of task-level data to confirm their estimation method. This highlights the fact that data acquisition is difficult and new, curated datasets should be developed. This scarcity has forced researchers to operate with limited data or create new and edited datasets to ensure the validity and applicability of their models [23]. In order to develop successful estimation models, it is important to determine which factors or drivers of effort have significant effect on the work that is required. Technical factors which are typically utilized such as complexity of tasks, scope of work, and uncertainty of requirements [29, 30]. A Fuzzy AHP framework was adopted by Abusaeed et al. [31] to rank cost factors, where technical complexity and skill of developer

were the most important. Chahar and Bhatia [32] defined project scope and team capability as key elements in software testing effort.

Besides the technical factors, dynamics of the team are also considered to be significant drivers. A comprehensive study by Sunil Kumar Suvvari and colleagues [12] demonstrated that estimation accuracy is strongly correlated with team size, cohesion and psychological safety. This is consistent with the results of Butt et al. [10], who proposed an approach that utilized the expertise of developers to enhance cost estimation. To model these complex effort drivers and predict results researchers started using more advanced ML methods, leaving behind the traditional statistical models. Numerous ML algorithms have been used for effort estimation. Initial research typically compares traditional algorithms such as Decision Trees, Random Forest and Gradient Boosting [33, 34]. Gultekin and Kalipsiz [35] have demonstrated high accuracy with such models as Support Vector Regression (SVR) and Multi-layer Perceptron (MLP). Other emerging methods involve the application of Extreme Learning machine (ELM) that was revealed to be superior when compared to the traditional models [36] and fuzzy logic systems to deal with the existing uncertainty in estimation [28, 37, 38]. Favero et al. [11] and Atoum et al. [27] used pre-trained embedding models such as BERT and FastText to predict effort using text. But replication studies have demonstrated these complex models do not necessarily do better than simple baselines and may not generalize well [13, 39], indicating that structured features are highly important.

By acknowledging that no single model is universally superior [40], researchers have started paying more attention to ensemble learning which is a technique to combine predictions of many models in order to get a higher accuracy and strength [41]. Jadhav and Shandilya [42] discovered that Random Forest, which is an ensemble, was always better than the other algorithms. K. Eswara et al. [17] used ensemble learning along with feature elimination to acheive considerable improvements. Ensemble frameworks have been suggested in several studies. Kaushik et al. [18] proposed the SENSE model, a stacking ensemble of SVR, LightGBM and other learners, which has shown high performance on various datasets. Likewise, Alsaadi et al. [15] designed an ensemble model tailored to novice agile teams with a Mean Absolute Error (MAE) of 0.78 and a significant increase in team confidence. The combination of various computational paradigms is also evidenced by hybrid models, like the ANFIS-EEBAT approach by Arora et al. [9] that combines neuro-fuzzy systems with optimization algorithms. This overview of the recent literature identifies three important issues: (1) effort estimation progress is undermined by a lack of high-quality, user-story-level data; (2) there is a complex set of effort drivers that should be accounted to estimate the effort accurately; and (3) ML and in particular, ensemble models, have shown the most potential to enhance the accuracy of prediction. Table 1 provides a detailed overview of some important studies, their methods, data sets and performance results.

Methodology

To fill the research gaps of literature this research followed Design Science Methodology (DSR). DSR is a methodology focused on creating innovative solutions to real-world problems by designing and evaluating artifacts e.g., models, methods or systems. It combines rigor and relevance, producing prescriptive knowledge that bridges theory and practice. DSR emphasizes iterative development, collaboration and the evolution of

Table 1 Summary of recent studies on effort estimation techniques in agile software development

Sr	Paper title/author(s)	Method/model	Dataset	Performance/key finding	Proposed system
1.	Ersoy et al. [28]	COSMIC Function Point (CFP), Pearson Correlation	Industrial case study (330 tasks from a FinTech company)	Strong correlation between CFP and actual effort ($r > 0.75$).	No
2.	Alsaadi et al. [15]	Ensemble Model (Extra Trees, KNN, MLP)	140 user stories from novice Agile teams	MAE: 0.78, RMSE: 1.15.	No
3.	Kaushik et al. [18]	Stacking Ensemble (SENSE Model)	6 public datasets (Nasa93, Kemerer, etc.)	High accuracy ($R^2 > 90\%$) across multiple datasets.	No
4.	Arora et al. [9]	Hybrid: ANFIS + energy-efficient BAT algorithm	Zia & ZKmS Agile datasets	High accuracy reported (99.93% on ZKmS); MMRE: 1.518.	No
5.	Atoum et al. [27]	Pre-trained Word Embeddings (FastText) + SVM	Deep-SE dataset (23,313 user stories)	MAE: 2.18, RMSE: 2.92; good for small datasets.	No
6.	Tawosi et al. [13]	Replication of GPT2SP (Transformer-based DL)	New dataset of 31,960 user stories	Found that GPT2SP failed to outperform simple baselines.	No
7.	Lavingia et al. [7]	Random Forest, SVR, etc.	NASA project data (benchmark)	Random Forest performed best ($R^2 = 0.79$).	No
8.	Shidiq et al. [33]	Gradient Boosting with PCA	Scrum project dataset	Gradient Boosting with PCA gave the best performance.	No
9.	Gultekin & Kalipsiz [35]	SVR, MLP, Random Forest, Gradient Boosting	JIRA, MongoDB, Apache datasets	High accuracy (99.8% with Gradient Boosting on MongoDB).	No
10.	Kumar [36]	Extreme Learning Machine (ELM)	Public datasets	ELM showed ~ 15% improvement in accuracy over traditional models.	No
11.	Sánchez et al. [34]	Decision Tree, Random Forest, AdaBoost	Agile project data	Random Forest achieved high accuracy of 96.92%.	No
12.	Turic et al. [30]	Bayesian Network Model	Agile project data	Achieved over 90% accuracy, good for handling uncertainty.	No
13.	Karna et al. [25]	k-Nearest Neighbors (KNN)	Local data from a single large telco project	Predictive model improved accuracy over expert assessors.	No
14.	Jadhav & Shandilya [42]	Comparative analysis of 8 ML algorithms	12 public datasets	Random Forest showed superior performance in accuracy.	No

design knowledge to address complex, dynamic challenges in information systems and beyond [43]. There are three phases in DSR methodology as shown in Fig. 2.

a) The initial step was problem identification and motivation to explore and define the research question. This phase involved three important activities: Literature review and rationale understanding to find out research gaps in effort estimation in agile software projects. Formulation of research questions to establish the fundamental issues covered by the research. Definition of research objectives to determine measurable goals to create an effective solution.
b) The second phase prototype design and development involved three activities: Data collection to obtain the relevant project metrics to train the ML model. To design a

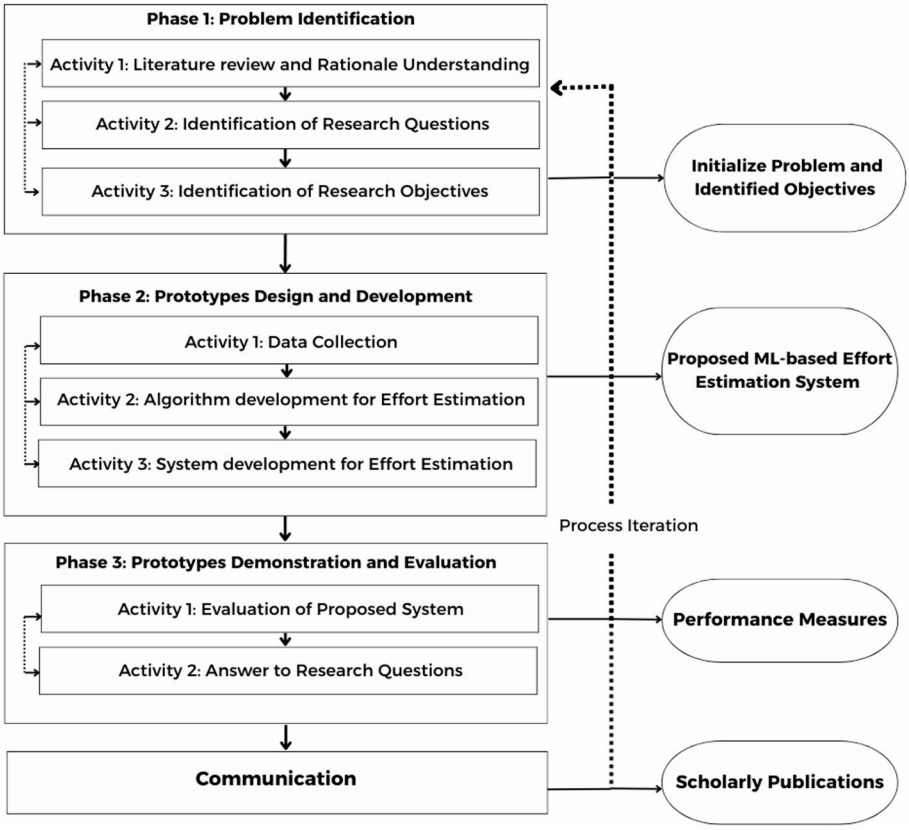


Fig. 2 DSR methodology applied in the development and evaluation of the proposed ensemble model. Solid arrows represent the sequential flow of activities and corresponding outputs, while dotted arrows represent process iterations and feedback loops between phases

ML based model to predict software project effort. System development for effort estimation to deploy the algorithm into a working prototype that helps project managers to make data-driven decisions.

c) The third phase prototype demonstration and evaluation is divided into two activities: Evaluation of the proposed system to assess its accuracy and reliability. Working of model was evaluated by performance measures. Answer to research questions by validating the system to determine whether it achieves the objectives outlined in the Phase 1 to confirm its contribution to the improvement of the effort estimation.

The final phase of the DSR methodology is Communication, which involves disseminating the research findings, the technical architecture of the developed artifact, and its evaluation results to the academic community and industry stakeholders.

Problem identification phase

A systematic problem identification phase was carried out in order to define the direction of this research. The objective was to understand the current problems of accurately estimating user story effort in agile methodologies, specifically scrum. The thorough comparison of the current estimation techniques such as expert-based, ML and hybrid was already provided in Sect. 2: Literature Review, and where different models and their limitations were critically discussed. Additionally, the reason to enhance the accuracy of estimations with the help of ML was employed to generate targeted Research Questions

that have been clearly defined in the Introduction section. The aim of these questions is to investigate how various effort drivers affect the accuracy of estimation and whether ML-based models can be more effective than the traditional ones. The literature review and the research questions collectively presented a clear understanding of how the research problem was identified and the rationale of the proposed ensemble-based estimation model.

Prototype design and development

The second phase, Prototype Design and Development, integrates three core activities: data collection to construct the high-fidelity dataset, algorithm development to implement the novel stacking-based ensemble architecture, and system development to operationalize the model into a functional web-based prototype for real-time estimation support.

Data collection

Data is important in estimating user story effort based on historical data. Data in this field is either newly gathered or obtained from literature. However, the literature shows that there are few user story-level datasets relative to the project-level datasets. The data used in this context can be divided into two groups structured and unstructured. Structured data is normally stored in relational databases and has categorical and numerical features. Unstructured data on the other hand is mostly textual description of projects or user stories that may include programming commands. Data collection presents a number of challenges despite its significance:

1. **Data Privacy**: Effort estimation datasets usually contain personal information concerning projects, teams and software features. Many organizations are not ready to provide such data to researchers due to privacy issues.
2. **Data Availability**: Privacy restrictions significantly limit the number of publicly accessible datasets. Although some organizations collaborate selectively with researchers, many declines to publish their data.
3. **Data Size**: The volume of data also presents a challenge. Unstructured datasets, especially from open-source repositories, tend to be large e.g., datasets with over 23,000 records [44]. In contrast, structured datasets, typically sourced from proprietary projects, are relatively small, ranging from 160 to 503 records.

One of the main goals of this research was to create a strong and thorough dataset for estimating user story effort. Publicly available datasets offer limited research value, as they predominantly focus on project-level estimation or contain only a small number of user stories. Therefore, a team gathered wholesale data in a coordinated way to serve the research community. Data collection phase consist of following 3 steps as shown in Fig. 3.

a) Identification of Effort Drivers

The project team made it a priority to find and analyze the primary aspects that have the biggest impact on the effort needed in user story development. A dual methodology was used, drawing from both academic learning and practical data from the industry.

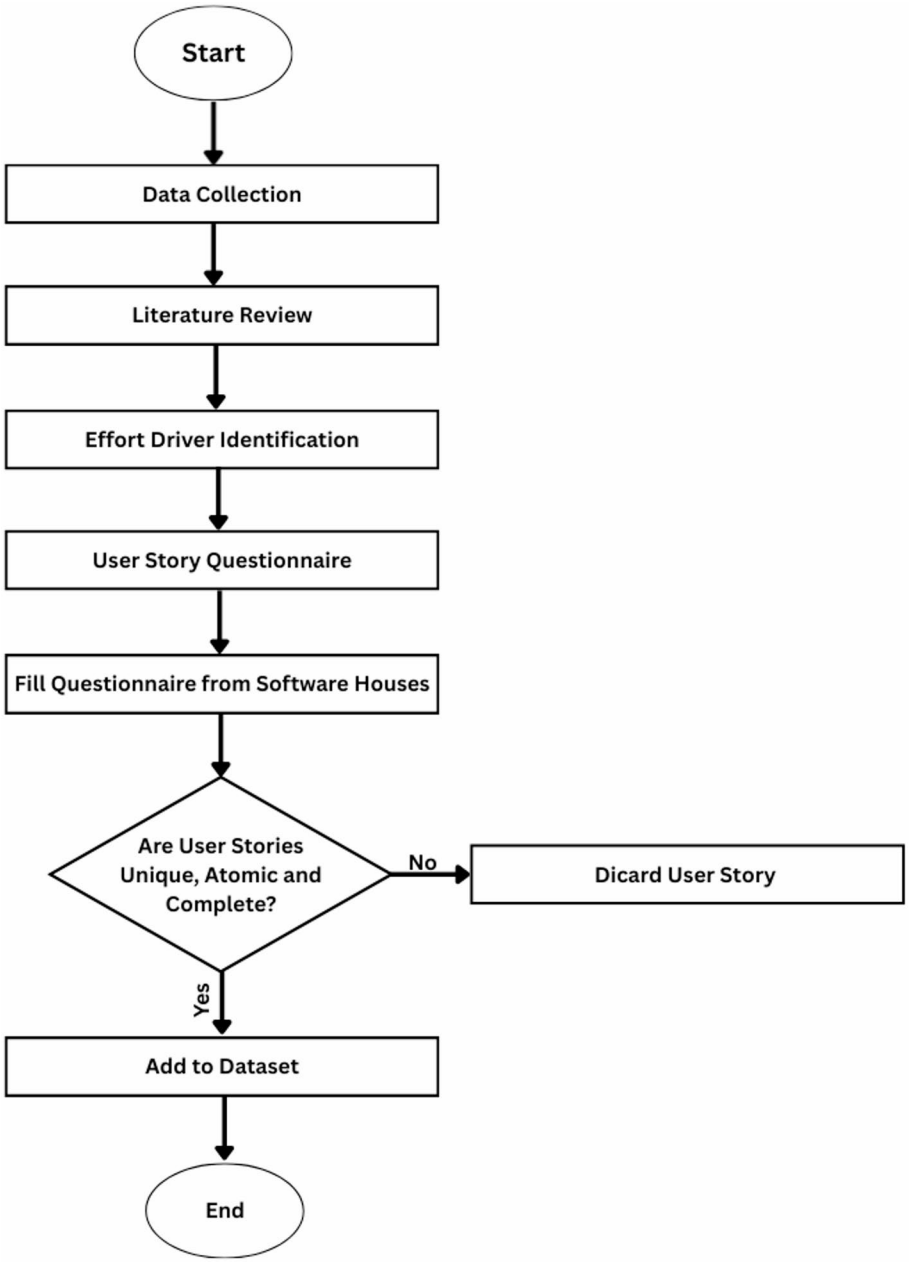


Fig. 3 Data collection phase

Literature Review: A thorough review of published studies, conference papers and suitable books in software engineering and agile techniques was done. The primary objective of this literature search was to identify and synthesize factors that have been frequently utilized as effort drivers across existing research studies. Using this method, this research identified a list of 12 effort drivers, each related to different aspects of software development effort [2].

Industry Insights from Software Houses: To add real-world experience to the academic research several software houses were visited that were actively practicing Scrum and other agile methodologies. After discussion with scrum team and analyzing projects that followed scrum methodology this research found an effort driver named as

"Requirements Volatility". While this driver is often a concern in real projects, it is not as regularly identified with data collection by academics.

All the findings from the review and consultations were put together to form a list of thirteen effort drivers. As recommended in agile estimation, Story Points were given to the user stories and the Fibonacci sequence (e.g., 1, 2, 3, 5, 8 and 13) was used to measure both the estimated and actual effort. Distribution of effort across different drivers is shown in Fig. 4.

There are thirteen identified effort drivers included in our dataset, as shown below:

1. **Experience**: The software development experience of team members is widely recognized as a critical factor in effort estimation across numerous studies. Generally, developers with greater experience tend to require less effort to complete user stories than those with limited exposure. In this dataset, the user stories were sourced from industry-level projects executed within professional software houses. The participating team members possessed varying levels of experience, ranging from 10 to 40 months, reflecting a mix of early-career professionals and moderately experienced developers.
2. **Priority**: This refers to how critical or urgent a user story is in the context of the overall project. Generally, stories with higher priority are addressed first. In this dataset, priority is represented as an integer scale, where '1' indicates the highest priority, '2' the next highest and so on. The priority values range from 1 to 13.
3. **Size**: The size of a user story has a direct impact on the development effort required. Larger stories typically demand more time and resources than smaller ones. For this purpose, five-point size scale is used, where '1' indicates the smallest and '5' the largest. In this dataset, size '3' is the most frequently occurring, while only a few user stories are of size '5'.
4. **Complexity**: Understanding how complex a user story is plays an important role in estimating effort. Complexity is influenced by several factors including interdependencies with other stories, the need for additional research and the absence of required technical skills. Like size, complexity is also measured on five-point scale.

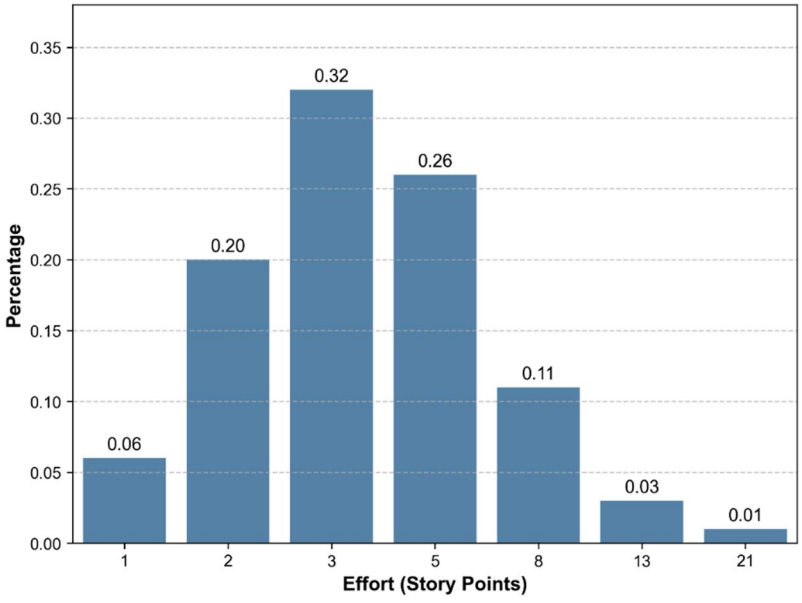


Fig. 4 Effort distribution across effort drivers

Most user stories in the dataset fall within complexity levels 2 or 3, while only a few reaches level 5.

5. **Number of Tasks**: This factor represents the number of distinct sub-tasks associated with each user story. A higher number of sub-tasks typically corresponds to an increase in the overall development effort required. In this dataset, participants reported the exact number of sub-tasks per user story as a numeric value, ranging from 1 to 6.
6. **Team Members**: This represents the number of development team members. The numbers of team members in the dataset are 1, 2 and 3. However, having 3 team members involved in the development of user stories is the most frequent.
7. **Programming Languages**: The choice of programming language can also affect development effort. User stories in this dataset were developed using six languages: C#, Java, Swift, Python, JavaScript and HTML. Java and Python was the most widely used, appearing in over half of the user stories.
8. **Platform**: Platform refers to the target environment for the user story. Six platforms are represented in this dataset: Android Studio, iOS, Unity, Python,.NET and Google Colab. Android Studio was the dominant platform, used in over 125 user stories.
9. **Database System**: The dataset contains four database system: Firebase (the most frequent), SQL, Unity and Vuforia. Some stories did not involve any database at all.
10. **External Hardware**: This measure whether the story involves interaction with external hardware such as cameras or wearable devices. This is recorded as a binary value (Yes/No). Most user stories did not require any external hardware.
11. **Related Technologies**: This represents the advanced technologies or functional features used in implementing the user story. While some studies focus on functional features only, this dataset incorporates technologies such as Artificial Intelligence, location-based services, Blockchain, Gamification, Voice Recognition. These technologies, observed in senior project documentation, often demand extra effort especially from less experienced developers.
12. **Development Type**: This indicates the category of development work involved. The dataset includes four types: mobile app development, web development, model development and database development. Among these, web development was the most common.
13. **Requirement Volatility**: Reflects the degree of change in a user story's requirements. It is categorized into three levels: Low (no or minimal changes), Medium (moderate modifications) and High (frequent or significant changes). High volatility often increases development effort due to rework and coordination overhead. This qualitative scale helps in assessing the impact of changing requirements on estimation accuracy.

b) Questionnaire Design

A detailed and structured questionnaire was designed in order to obtain data relevant to the identified 13 effort drivers in a systematic manner. This tool was created to provide consistency and completeness in documenting every user story attribute, such as the estimated and actual effort in Fibonacci story points, in accordance with the standard agile estimation practices. The process of data collection has been carried out in a series of coordinated steps and these are as follows:

1. Selection of Participating Organizations: Software development companies that had long been practicing Scrum were selected as potential contributors. These organizations were selected on the basis of their active involvement in agile software development and project documentation.
2. Questionnaire Distribution: The questionnaire was distributed to relevant individuals in the chosen organizations after confirmation of their participation. These were project managers, Scrum masters and senior developers. The questionnaire was accompanied by detailed instructions to facilitate clarity in understanding and uniformity in the answers.
3. Data Compilation from Real Projects: The respondents were asked to fill the questionnaire with user stories sourced from recently completed or well documented projects. This strategy has been used to ensure that the data gathered represented real development situations and realistic efforts. Only user stories that had traceable estimate and actual effort values were gathered, which ensure relevancy and integrity of data.

This approach allowed the creation of a high quality dataset, based on realistic software development scenarios. This process is well-structured and rigorous, which helps to support the validity of the dataset, and it becomes a reliable source of research in of agile effort estimation.

c) Dataset Quality Assurance

A systematic quality assurance method was used to ensure that the dataset could be used to estimate the effort. The user stories and their related attributes were assessed against the following key criteria:

1. **Atomicity**: The user story should describe one well-defined functionality. Multi-featured stories were either refined or dropped.
2. **Uniqueness**: User stories that were duplicated or overlapping were also eliminated so that no two consecutive requirements were represented by the same story.
3. **Completeness**: Although complete project coverage was not required, the user stories submitted must reflected a sensible and understandable part of the project, with well-defined boundaries.

After this review 160 user stories of 36 software projects were left. Every record carries values of all 13 effort drivers and the actual effort in Fibonacci story points. The curated dataset is supposed to facilitate additional research in agile effort estimation as it provides a high-quality and structured resource at the level of the user story.

Algorithm for effort estimation

Following are steps for designing algorithm. Proposed model is shown in Fig. 5.

a) Data Preprocessing

In the initial phase of the algorithm development, data preprocessing is carried out to ensure that the dataset is clean, consistent and suitable for ML models. This step involves handling both categorical and numerical features appropriately. The transformation of the 160 user stories into a machine-learnable format was a direct extension of the data collection and quality assurance phases. Each of the 160 retained stories was mapped

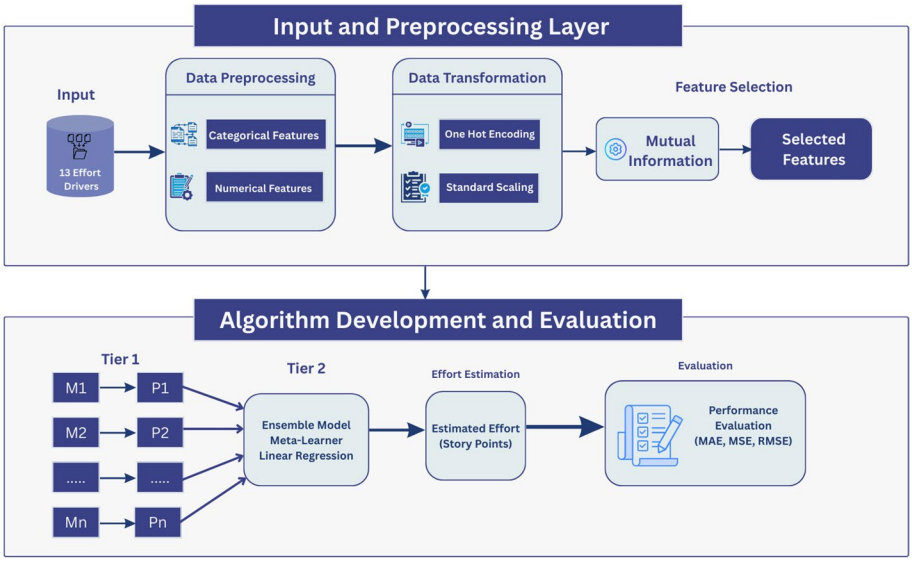


Fig. 5 Proposed ML based ensemble model architecture

to the 13 granular effort drivers identified through expert consultation and industrial review.

For categorical data, One-Hot Encoding (OHE) was used to convert the 160 user stories into numerical format suitable for training models [45]. This technique creates a new binary column for each category in a feature, so that the model cannot assume that there is an incorrect ordinal relationship between categories. For example, a categorical feature 'Technology' with values like AI, Blockchain and Computer Vision is converted into three different binary columns as follows: Technology_AI, Technology_Blockchain, and Technology_ComputerVision. Through this process all categorical effort drivers in the dataset such as Programming Languages, Platforms and Database Systems were expanded into a high-dimensional feature space. Consequently, the initial 13 raw effort drivers were converted into a final processed dataset which contained 35 numerical features. Such tabular format is required by the algorithms such as XGBoost and random forest, as they can determine accurate split thresholds and adequately model the nonlinear relationship that exists between the input drivers and the estimated effort story points.

In case of numerical data, StandardScaler is used to standarize the features. This process transforms the input variables by subtracting the mean and scaling them to unit variance, resulting in a feature set with a zero mean and a standard deviation of one. This procedure ensures that all numerical features are brought to a uniform scale, preventing features with larger raw magnitudes from disproportionately influencing the model's performance solely due to their numerical range. As an example, when an Experience_Years feature has the values 2, 5 and 10, the StandardScaler changes them to values with a mean of 0 and a standard deviation of 1 (almost – 1.14, 0.00, and 1.14), assuming a sample mean of 5.67 and a standard deviation of 2.90). These preprocessing methods help in transforming of the raw input data to a structured and normalized form, for other modeling processes.

b) Feature Selection

Following the preprocessing, feature selection is done to increase the performance of the model and minimize overfitting. Mutual Information (MI) has been selected as the core method of measuring feature relevance and informing selection process in this study [46]. In contrast to basic correlation coefficients, which can only identify linear associations between input variables and the target variable, MI can measure both linear and non-linear associations. It is especially relevant because the factors that affect software effort can be complex and non-linear. MI measures how much information is received about one variable (e.g., effort) by observing the other variable (e.g., complexity) and gives a direct measure of feature relevance. An increase in MI scores shows more robust dependencies and, therefore, higher predictive ability. Features that have greater MI scores are more related to the target variable and therefore are more informative to the predictive model.

However, as a filter-based feature selection method, MI effectively provides a ranking of feature importance but does not inherently identify the optimal subset size (K) required to maximize model precision. To ensure the selection process was grounded in empirical evidence rather than arbitrary thresholds, the MI-based ranking served as the foundation for a subsequent Sensitivity Analysis. In this optimization phase, various feature counts were incrementally tested ranging from K = 3 to K = 12 to determine the specific subset size that minimizes the bias-variance tradeoff. This systematic evaluation ensured that the final selection of the features was based on a quantitative reduction in prediction error, effectively preventing the inclusion of stochastic noise while retaining maximum predictive information.

c) Effort Estimation models

In the third phase of the algorithm, multiple ML models are trained to evaluate their individual predictive performance and to later combine them into a more robust ensemble model. A total of eight regression algorithms were employed, representing a diverse set of learning paradigms to capture different data patterns and relationships. These models include.

1. **Linear Regression**: Linear Regression is a fundamental supervised learning algorithm that models the relationship between multiple input features and a continuous target variable using a **Multiple Linear Regression (MLR)** model. Given that this study utilizes 13 distinct effort drivers, the relationship is represented by the following equation:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_n x_n + \epsilon$$

where:

- Y is the predicted effort (Story Points).
- β_0 represents the y-intercept (constant).
- $x_1, x_2, \ldots x_n$ represent the individual input effort drivers (features).
- $\beta_1, \beta_2, \ldots \beta_n$ are the regression coefficients for each respective feature.
- ϵ denotes the random error term.
- It's simple, interpretable and works well when data follows a linear trend. The model minimizes the mean squared error between predictions and actual values. However, it struggles with nonlinear relationships unless polynomial features are added.

2. **Decision Tree Regressor**: Decision Trees split data into branches based on feature thresholds, making predictions by averaging values in terminal leaves [47]. They handle nonlinear data well and require little preprocessing but are prone to overfitting. Hyperparameters like max_depth control complexity.
3. **Random Forest Regressor**: Random Forest is an ensemble method that builds multiple decision trees on random subsets of data and features, then averages their predictions. This reduces overfitting and improves generalization. It's robust, handles missing values well, and provides feature importance scores [47].
4. **Extra-Trees Regressor (Extremely Randomized Trees)**: Extra-Trees is a variant of Random Forest that randomizes both data sampling and feature split thresholds. This makes training faster but can increase variance. It's useful for large datasets where computational efficiency matters [47].
5. **Support Vector Regressor (SVR)**: SVR uses Support Vector Machines for regression by fitting the flattest possible "tube" (epsilon-insensitive band) around the data. Kernel tricks (e.g., RBF) allow it to model nonlinear relationships. Works well in high dimensions but can be slow for large datasets [47].
6. **K-Nearest Neighbors (KNN) Regressor**: KNN predicts by averaging the target values of the k closest training examples. It's simple and adapts to nonlinear trends but suffers from the "curse of dimensionality" and requires careful choice of k and distance metrics [47].
7. **XGBoost Regressor**: XGBoost is a scalable gradient-boosted tree algorithm that sequentially corrects errors from previous models. It includes regularization to prevent overfitting and handles missing data automatically. Often a top choice for competition-winning performance [47].
8. **Multi-Layer Perceptron (MLP) Regressor**: MLPs are feedforward neural networks that learn hierarchical feature representations. They excel at complex, high-dimensional data but require extensive hyperparameter tuning (layers, activation functions) and large datasets to avoid overfitting [47].

d) Ensemble model

Ensemble learning is a ML technique that combines the predictions of multiple individual models to improve overall performance [47]. By leveraging the strengths of different models and reducing their individual weaknesses, ensemble methods often achieve higher accuracy, robustness and generalization compared to single model. There are several types of ensemble methods, including.

1. **Bagging**: Combines models trained on different subsets of the data.
2. **Boosting**: Sequentially builds models where each new model corrects the errors of the previous one.
3. **Stacking**: Uses a meta-model to combine the predictions of multiple base models.

This study employs a Stacked Generalization (Stacking) architecture to optimize the final effort prediction. In this hierarchical framework, the outputs from the three top-performing base-learners Extra Trees, XGBoost, and Random Forest serve as input features for a Tier-2 Meta-Learner Linear Regression. The meta-learner is trained to learn the optimal regression coefficients for each base-model's prediction, effectively minimizing the overall variance and bias. This approach ensures that the 'weights' assigned to

each individual regressor are determined mathematically through the learning process, leading to a more robust and adaptive estimation compared to static averaging methods.

e) Pseudo code of model

Input: Size, Complexity, Priority, Experience, Related technologies, no of tasks, Team members, Programming languages, Development type, Database system, External hardware, Related Technologies, Platform, Requirement volatility.
Output: Estimated Effort

BEGIN

1. Get **Size, Complexity, Priority, Experience, Related technologies, no of tasks, Team members, Programming languages, Development type, Database system, External hardware Related Technologies, Platform, Requirement volatility** from **scrum_master**.
2. **effortEstimationData** = Select effort drivers(base_features) and actual efforts (target_column) from **EffortEstimationTable**
3. Where **size=?, complexity=?, experience=?, priority=?, no of tasks=?, Team members=?, Programming languages=?, Platform=?, Development type=?, database system=?, Requirement volatility =?, External hardware=?** and **related technologies=?**
4. processedData = PreprocessData(**effortEstimationData**)
5. Apply StandardScaler to Numerical_Columns
6. For each **numerical_column**
7. Calculate mean = average of all values in column
8. Calculate std_dev = standard deviation of all values in column
9. For each value in column
10. **scaled_value** = (value - mean)/std_dev
11. Replace original value with **scaled_value**
12. Apply **One-Hot Encoding to Categorical_Columns**
13. For each **categorical**_column
14. Identify all unique **categories** in the column.
15. Create a new **binary**_column for each unique category.
16. For each row
17. Set the new **binary**_column to 1 if the row belongs to that category.
18. Set the new **binary**_column to 0 if the row does not belong to that category.
19. Drop the original **categorical**_column after encoding.
20. Return the transformed dataset with new **binary_columns**.
21. Apply **Mutual Information (MI) Feature Selection**:
22. Calculate **MI scores**:
23. X = processedData.drop(columns=['effort']) (Features)
24. y = processedData['effort'] (Target)
25. **mi_scores** = **mutual_info_regression**(X, y)
26. **mi_scores_df**['Feature'] = X.columns
27. **mi_scores_df**['MI Score'] = mi_scores
28. **selectedFeatures** = mi_scores_df.nlargest(K, **'MI Score'**)['Feature'].tolist()
29. **Define** Base_Models B=[Extra Trees, XGBoost, Random Forest]

30. **Define** Meta_Learner M=Linear Regression
31. **Function** StackingEnsembleEstimation(**selectedFeatures**, B, M):
32. **Initialize** Tier1_Outputs = []
33. **For** i=0 **to** 2 **do**:
34. Pi=BasePredict(B[i],**selectedFeatures**)
35. **Append** Pi **to** Tier1_Outputs
36. **End For**
37. FinalEstimatedEffort = **MetaPredict**(M, Tier1_Outputs)
38. **Return** FinalEstimatedEffort

END

System development for effort estimation

A functional prototype was developed to demonstrate the practical application of the proposed ensemble model. The system includes a web-based interface where users, such as Scrum Masters, can input the 13 effort drivers for a given user story. Upon submission, the backend algorithm processes these inputs and returns the estimated story points, serving as a real-time decision support tool. The operational workflow of the system is illustrated in Figs. 6, 7 and 8. Figure 6 shows the landing page, while Fig. 7 presents the initial user story entry point. Figure 8 displays the comprehensive input form where 13 effort drivers are selected, along with the final model-generated effort estimation result.

Prototype demonstration and evaluation

This phase includes evaluation of proposed algorithm and answer to research questions.

Evaluation of proposed system

This phase includes evaluating accuracy and reliability of the predictive models. To train and evaluate the models, the dataset was divided into two groups, with 80% of the data (128 samples) used in the training step and the remaining 20% (32 samples) used in independent testing to evaluate the predictive capabilities of the models. At this stage, the performance of the model is evaluated based on well-known metrics of errors to measure the discrepancy between actual and predicted values. Following Error metrics are used to assess model.

a) Mean Absolute Error (MAE)

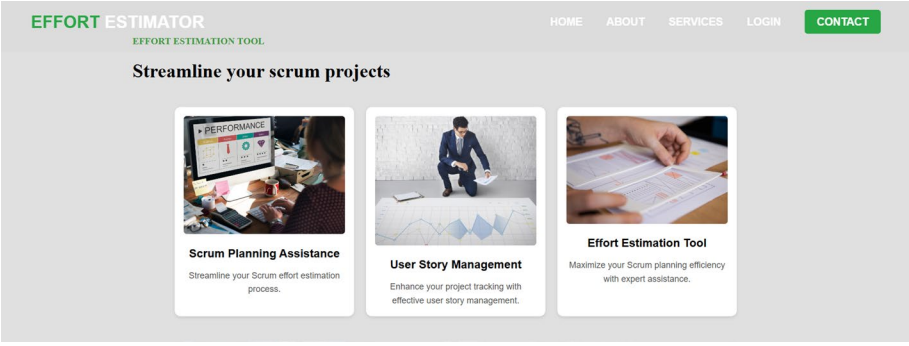


Fig. 6 System landing page providing access to the system

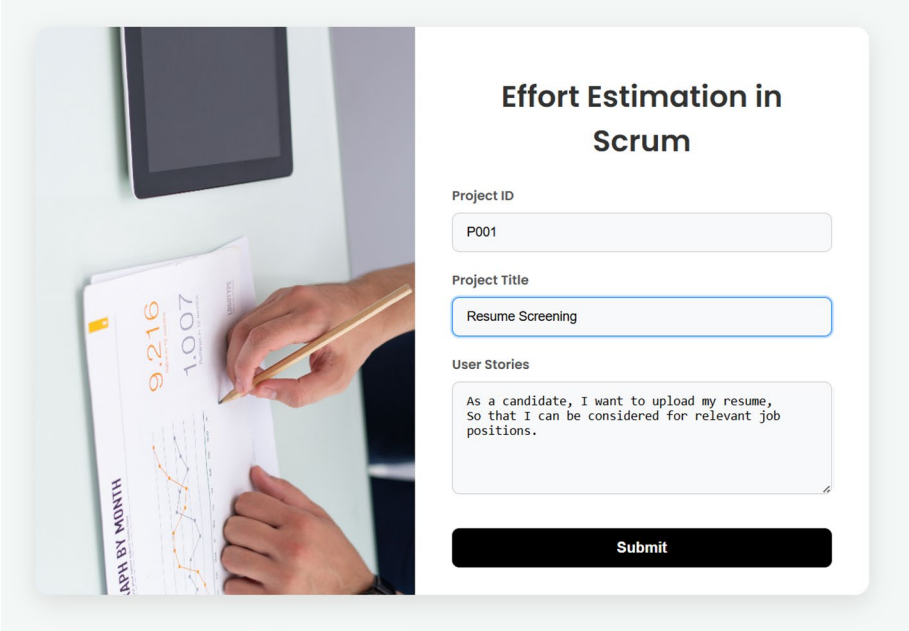


Fig. 7 Input interface where practitioners provide user stories for estimation

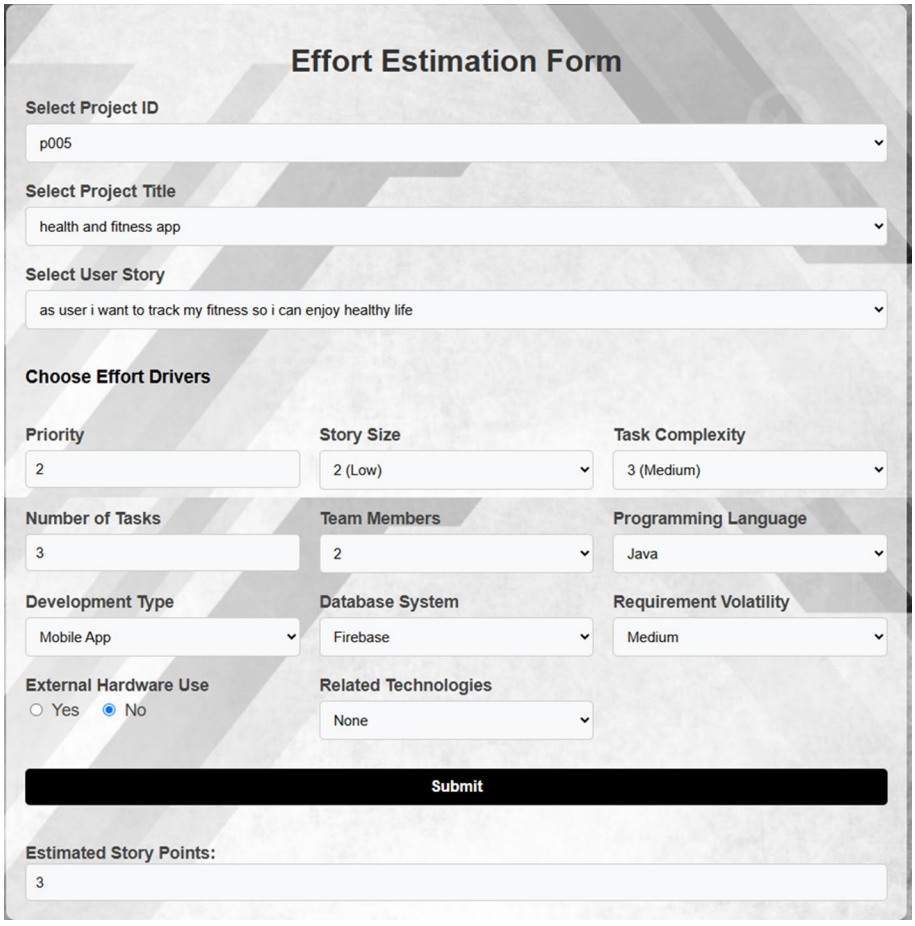


Fig. 8 Effort estimation output by the proposed system

MAE measures the average magnitude of errors between predicted and actual values, without considering their direction [48]. It provides a straightforward interpretation by indicating how far, on average, the predictions deviate from the actual outcomes. Unlike metrics that involve squaring the errors, MAE treats all deviations equally, making it less sensitive to outliers. This metric is particularly effective when errors are expected to be uniformly distributed. The formula of MAE is:

$$\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n} | \, y_i - \hat{y}_i \, |$$

where

n is the total number of user stories in the test set.

y_i represents the actual effort (Actual Story Points) for the i-th user story.

$\hat{y}_i$represents the predicted effort (Estimated Story Points) generated by the model for the i-th user story.

$y_i - \hat{y}_i$ represents the prediction residual.

b) Mean Squared Error (MSE)

MSE calculates the average of the squared differences between actual and predicted values [49]. By squaring the errors, MSE assigns greater penalty to larger errors, making it suitable for highlighting models that make significant prediction mistakes. The formula for MSE is:

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n} (\, y_i - \hat{y}_i)^2$$

c) Root Mean Squared Error (RMSE)

RMSE is calculated by getting the square root of MSE, thereby returning the error to the same unit as the target variable [48]. RMSE gives the advantages of MSE with more interpretable values. The formula of RMSE is:

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n} (\, y_i - \hat{y}_i \,)^2}$$

Answer to research question

The research question that has been developed at the initial stage of the current study What factors affect accuracy in Scrum effort estimation? is answered in the data collection and problem identification phases of the study. This study identified and confirmed thirteen major effort drivers that affect the accuracy of Scrum effort estimation through a dual approach that incorporated a comprehensive literature review and a direct industry consultation. These factor consider various dimensions of a project such as task-specific attributes (e.g., Complexity, Size, Number of tasks), team dynamics (e.g., Team Member Experience) and project environment (e.g., Development Type, Requirement Volatility). The breakdown and rationale behind each of these drivers can be found in Sect. 3.2.1, titled Identification of Effort Drivers.

Regarding the second research question, "Does estimation accuracy in Scrum-based projects improve through the application of ML techniques?", the empirical evaluation conducted in this study provides a clear and affirmative answer. The application of advanced ML models, specifically the proposed novel stacking-based ensemble, significantly improves estimation precision. By hierarchically integrating Extra Trees, XGBoost, and Random Forest as Tier-1 base learners and utilizing Linear Regression as the Tier-2 meta-learner, the model achieved an MAE of 0.51 and an RMSE of 0.70. This hierarchical architecture outperformed all eight individual ML models tested, confirming that a tiered stacking approach which mathematically synthesizes the strengths of heterogeneous base learners provides a more robust, precise, and reliable solution for effort prediction in professional Scrum environments. The comprehensive comparative metrics and visual analysis are detailed in Sect. "Results".

Results

Eight ML models Extra Trees, XGBoost, Random Forest, Linear Regression, MLP, KNN, Decision Tree, and SVR were compared in this study to predict user story effort in Scrum projects. Although MI offered a strong relative importance ranking of the 13 identified effort drivers, it did not necessarily define the optimal number of features of maximum predictive accuracy. In order to overcome this, systematic sensitivity analysis was conducted, whereby feature subsets of K = 3 to K = 12 were tested. This assessment was an empirical optimization step, necessary to identify the specific number of top-ranked features, which reduce the bias-variance tradeoff and avoid the inclusion of stochastic noise. This procedure made sure that the final feature set was chosen on the basis of quantitative evidence instead of arbitrary limit.

The experiments showed that all individual models gave the lowest error rates when a subset of 11 features was used, namely: priority, size, complexity, number of tasks, development type, related technologies, programming languages, database system, requirement volatility, team members, and external hardware. For example, Extra Trees achieved its highest performance with an MAE of 0.52, MSE of 0.71 and RMSE of 0.84 when K = 11. On the same note, XGBoost and random forest demonstrated considerable increase in accuracy at this threshold. The performance of the eight models on all feature subsets is provided in Tables 2 and 3, and 4, and the performance of the models at the optimal 11-feature mark is plotted in Figs. 9 and 10, and Fig. 11. These results confirm that the more relevant drivers enable the models in capturing the intricate and non-linear trends of the agile effort estimation.

According to these benchmarking results, a new stacking-based ensemble model was proposed in order to exploit the complementary learning paradigms of the most

Table 2 Mean absolute error of individual models

Model	K = 3	K = 4	K = 5	K = 6	K = 7	K = 8	K = 9	K = 10	K = 11	K = 12
Extra trees	0.96	0.89	1.02	0.96	0.98	0.97	0.74	0.52	0.52	0.52
XGBoost	0.88	0.82	0.92	1.05	0.54	0.52	0.49	0.54	0.56	0.54
Linear reg	1.08	0.99	0.99	0.97	0.83	0.75	0.68	0.68	0.69	0.68
Random forest	0.88	0.85	0.87	0.97	0.95	1.11	1.11	0.69	0.66	0.69
MLP	0.99	0.97	1.03	0.86	1.00	0.87	0.84	0.75	0.71	0.79
SVR	0.90	1.02	1.02	0.99	0.73	0.74	0.74	0.74	0.73	0.74
Decision tree	1.01	1.01	1.01	1.07	1.06	1.20	1.25	0.76	0.76	0.76
KNN	0.75	0.97	1.08	1.01	0.74	0.69	0.72	0.73	0.78	0.71

Table 3 Mean squared error of individual models

Model	K=3	K=4	K=5	K=6	K=7	K=8	K=9	K=10	K=11	K=12
XGBoost	1.10	1.38	1.71	1.97	0.64	0.59	0.54	0.71	0.71	0.74
Extra trees	1.42	1.56	2.09	1.60	1.65	1.60	1.10	0.75	0.73	0.75
Linear reg	1.84	1.77	1.78	1.73	1.30	1.15	0.89	0.89	0.89	0.91
Random forest	1.07	1.03	1.15	1.85	1.91	1.93	1.97	1.07	1.04	0.99
MLP	1.56	1.51	1.63	1.35	1.66	1.48	1.53	1.16	1.43	1.02
SVR	1.90	2.13	2.13	2.00	1.49	1.48	1.55	1.55	1.54	1.53
Decision tree	1.41	1.41	1.41	1.78	1.73	1.98	2.09	1.63	1.63	1.63
KNN	0.90	1.45	1.95	1.84	1.15	0.93	1.03	1.04	1.00	1.27

Table 4 Root mean squared error of individual models

Model	K=3	K=4	K=5	K=6	K=7	K=8	K=9	K=10	K=11	K=12
XGBoost	1.05	1.17	1.31	1.40	0.80	0.77	0.73	0.84	0.84	0.86
Extra trees	1.19	1.25	1.45	1.27	1.28	1.26	1.05	0.87	0.86	0.87
Linear reg	1.36	1.33	1.33	1.32	1.14	1.07	0.94	0.94	0.94	0.95
Random forest	1.03	1.01	1.07	1.36	1.38	1.39	1.40	1.03	1.02	1.00
MLP	1.25	1.23	1.28	1.16	1.29	1.22	1.24	1.08	1.19	1.01
SVR	1.38	1.46	1.46	1.41	1.22	1.22	1.24	1.24	1.24	1.24
Decision tree	1.19	1.19	1.19	1.33	1.31	1.41	1.44	1.28	1.28	1.28
KNN	0.95	1.20	1.40	1.36	1.07	0.97	1.02	1.02	1.00	1.13

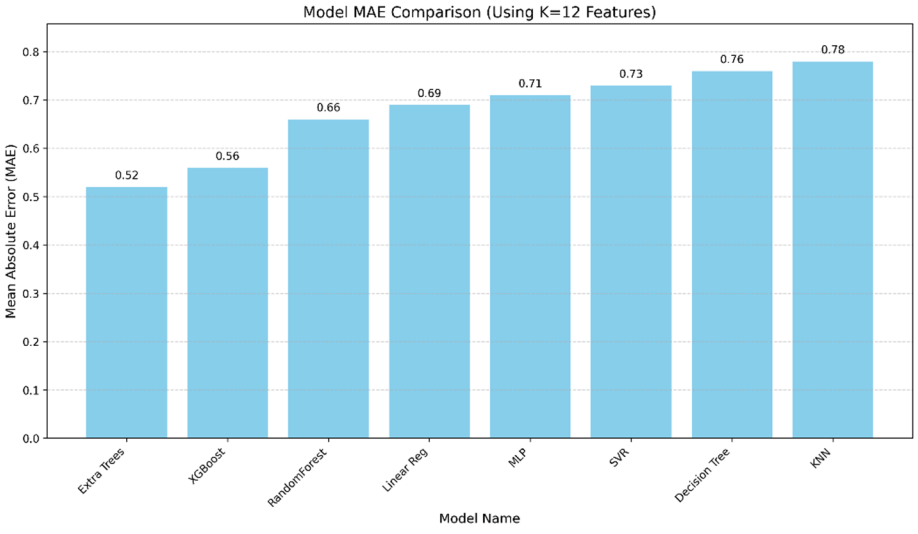


Fig. 9 MAE comparison of all models

effective regressors. Extra Trees, XGBoost and Random Forest were used in this hierarchical structure as Tier-1 base learners, and Linear Regression was used as the Tier-2 meta-learner. These particular base learners were chosen based on their varying mathematical backgrounds. Random Forest as well as Extra Trees offer strong generalization based on bagging and randomization to minimize the effect of variance whereas XGBoost employs a gradient boosting to minimize the bias by continuously adjusting the prediction residuals. This heterogeneity is a critical advantage; the ensemble architecture allows the gradient-boosting precision of XGBoost to capture complex dependencies that may be over-smoothed by traditional forest models, while the randomized

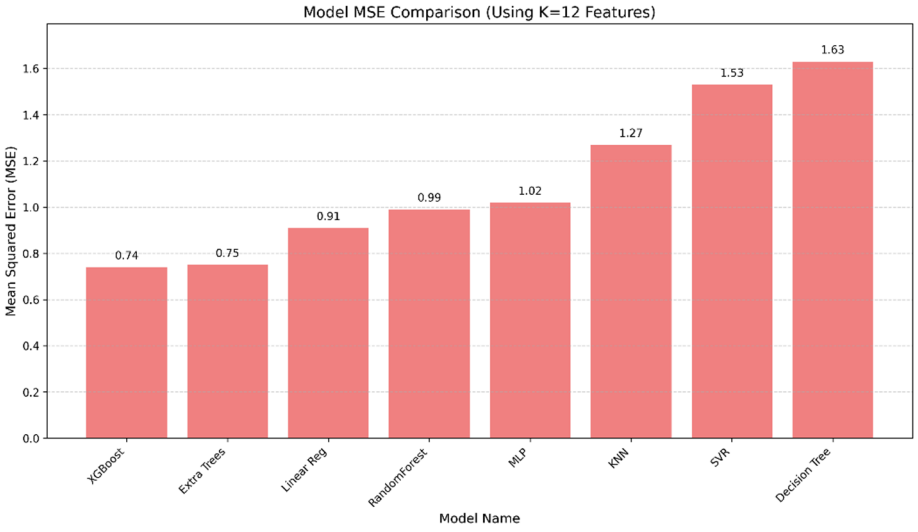


Fig. 10 MSE comparison of all models

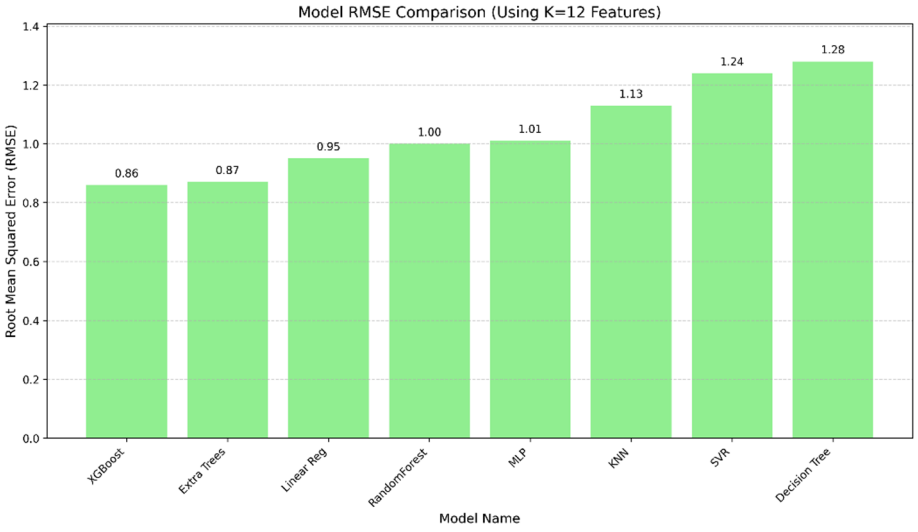


Fig. 11 RMSE comparison of all models

structure of Extra Trees helps mitigate the impact of noise within the 160 user-story dataset.

Rather than the simple averaging method, the Linear Regression meta-learner was also trained to learn the best coefficients of each base regressor output. By examining the error patterns of the base learners, the meta-model synthesize the predictions of the base learners, so that the weaknesses of one algorithm overcome by the strengths of the other. This advanced model-aggregation approach led to a substantial decrease in the overall prediction error, with the final MAE of 0.51, MSE of 0.49 and RMSE of 0.70 as reported in Table 5. While the stacking ensemble introduces a hierarchical training process, the computational overhead remains negligible for the structured Scrum dataset, with inference times maintaining real-time responsiveness. This marginal increase in complexity is justified by the superior bias-variance tradeoff and predictive stability illustrated in Fig. 12. Ultimately, the stacking approach successfully leverages the diverse

Table 5 Comparison of individual models with ensemble model

Model	MAE	MSE	RMSE
Extra Trees	0.52	0.73	0.86
XGBoost	0.56	0.71	0.84
Random forest	0.66	1.04	1.02
Linear regression	0.69	0.89	0.94
MLP	0.71	1.43	1.19
SVR	0.73	1.54	1.24
Decision tree	0.76	1.63	1.28
KNN	0.78	1.00	1.00
Ensemble model	**0.51**	**0.49**	**0.70**

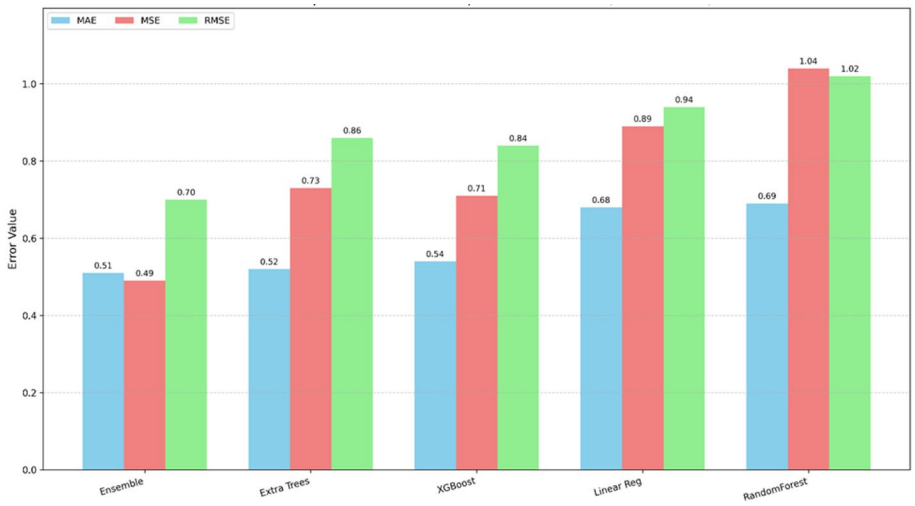


Fig. 12 Comparison of the ensemble model with the top-performing individual model

learning capabilities of the heterogeneous base models, providing a more reliable and generalized solution for effort estimation in professional Scrum environments than standalone individual models or simpler ensemble methods.

Comparison with state of art

ML techniques have been continuously optimized over the past several years to estimate effort in agile software development. Early research, including those by Shidiq et al. [33], used conventional algorithms like Decision Tree, Random Forest, and Gradient Boosting on Scrum datasets. Their study reported a high accuracy of Gradient Boosting with PCA with an MAE of 0.898 and an RMSE of 1.701, indicating that the error could be reduced further. Later more sophisticated architectures, like the transformer-based GPT2SP model, were introduced, but these were only found to perform better than simple baselines in 6 out of 16 agile projects, indicating that further efforts are required to improve generalizability [14]. Ensemble methods have also been of great interest; this is evidenced by the study on novice agile teams with an MAE of 0.78 and RMSE of 1.15 [2]. Regardless of these developments, other challenges have remained in the literature including overfitting and large prediction residuals.

Eight ML models such as Extra Trees, XGBoost, Random Forest, Linear Regression, KNN, SVR, and MLP have been evaluated rigorously in this study. The best-performing individual models scored MAE of 0.52, 0.54, 0.68, and 0.69 with the corresponding

Table 6 Comparison with state-of-the-art approaches

Sr	Paper title	Method/model	Dataset	Performance	Prototype
1	Software Effort Estimation Using Machine Learning Algorithms [50].	Random Forest, SVR, etc.	NASA project data (benchmark)	**SVR**: RMSE 323.61 **RF**: $R^2 = 0.2781$	No
2	Agile Effort Estimation: Have We Solved the Problem Yet? Insights From the Replication of the GPT2SP Study [14]	GPT2SP (Transformer-based DL)	16 Agile projects	Outperformed baselines in only 6/16 cases	No
3	Enhancing Software Effort Estimation through Reinforcement Learning-based Feature Selection [51]	RL-based Feature Selection (MARLFS)	Custom dataset with 31 selected features	MSE: 60.7 vs. 70.01 (expert judgment baseline)	No
4	Agile Software Development Effort Estimation based on Product Backlog Items [52]	ANN + NLP	Historical Product Backlog Items (PBIs)	Good training accuracy; poor validation (overfit)	No
5	Ensemble Effort Estimation for Novice Agile Teams [2]	Ensemble: Extra Trees, KNN, MLP	140 user stories from novice Agile teams	MAE: 0.78, RMSE: 1.15	No
6	An Efficient ANFIS-EEBAT Approach to estimate scrum project [53]	ANFIS + Energy-	Zia & ZKmS Agile datasets	R^2: 0.9993, MMRE: 1.518	No
7	Proposed model	Ensemble (Extra tree, Random Forest, Linear Regression, Xgboost)	160 user stories dataset	MAE: 0.51 MSE: 0.49 RMSE: 0.70	Yes

RMSE of 0.84, 0.86, 0.94, and 1.02. In order to further maximize the predictive accuracy, a new stacking-based ensemble model was constructed. Extra Trees, XGBoost and Random Forest are used as Tier-1 base learners in this hierarchical architecture, with a Linear Regression meta-learner used to synthesize the predictions of the base learners mathematically. The stacking method lets the model to learn the best contribution of each regressor and this leads to a very good estimation with a MAE of 0.51, MSE of 0.49 and RMSE of 0.70.

Table 6 shows the performance of the proposed stacking ensemble in comparison with the current state-of-the-art methods. Nevertheless, it is crucial to consider that since these studies use different datasets of different size and complexity, a direct head-to-head comparison is necessarily limited. Consequently, Table 6 results should be viewed as a contextual performance benchmark and no longer a statement regarding the overall superiority of the algorithms in all the datasets. The proposed model is placed in the context of the modern research environment by this comparison, proving its competitiveness and great reliability in the professional Scrum setting.

Conclusion and future work

This study addresses the critical challenge of accurate effort estimation in agile software development by focusing on user-story-level prediction. This study adopts the DSR methodology to propose and rigorously evaluate a novel stacking-based ensemble ML model. A high quality dataset of 160 user stories was built, including 13 important effort drivers identified from the academic literature as well as in real-world scrum projects. Eight ML models were trained and tested after the preprocessing of the data and a selection of features based on MI. The higher-level stacking ensemble architecture, combining Extra Trees, XGBoost, and Random Forest as Tier-1 base learners and Linear

Regression as the Tier-2 meta-learner, showed better results than single models with much lower values of MAE, MSE, and RMSE. These results confirm that hierarchical ensemble learning is effective in increasing the accuracy of estimations in dynamic agile settings. The proposed model significantly minimizes estimation errors and enhances the project planning, resource allocation, and team confidence. Moreover, a functional web-based front-end prototype was developed to operationalize the model and to act as a real-time decision-support tool. This provides an empirical, data driven alternative to the subjective experts based estimation, and can be helpful for less-experienced Scrum teams. The model can be incorporated into agile project management tools like Jira in future work so that real-time estimation of effort during sprint planning is possible. Furthermore, incorporating unstructured textual data through NLP techniques may further enhance prediction accuracy. A comparative analysis with deep learning models, such as LSTM networks, may also uncover more effective approaches for user story effort estimation.

Acknowledgements
Princess Nourah bint Abdulrahman University Researchers Supporting Project number (PNURSP2026R845), Princess Nourah bint Abdulrahman University, Riyadh, Saudi Arabia. Thanks to our families and colleagues, who provided moral support.

Author contributions
I.F. contributed to the conceptualization, dataset creation, algorithm design, and implementation. M.R. supported model development, experimentation, and result analysis. D.F. assisted with data preprocessing and the literature review. M.H. contributed to algorithm design, model evaluation, and led the main manuscript writing, in addition to leading the overall project and providing technical guidance. I.H.J. assisted with data analysis, experimentation, and result interpretation. A.T.Z. supported the literature review, manuscript editing, and formatting. All authors reviewed and approved the final manuscript.

Funding
Princess Nourah bint Abdulrahman University Researchers Supporting Project number (PNURSP2026R845), Princess Nourah bint Abdulrahman University, Riyadh, Saudi Arabia.

Data availability
The dataset will be shared upon reasonable request to the corresponding author.

Declarations

Ethics approval and consent to participate
Not Applicable.

Consent for publication
Not Applicable.

Competing interests
The authors declare no competing interests.

Received: 18 September 2025 / Accepted: 15 March 2026
Published online: 06 April 2026

References

1. Rubin KS. Essential Scrum: A practical guide to the most popular Agile process. Addison-Wesley; 2012.
2. Alsaadi B, Saeedi K. Ensemble effort estimation for novice agile teams. Inf Softw Technol. 2024. https://doi.org/10.1016/j.infsof.2024.107447.
3. Iqbal M, et al. Exploring issues of story-based effort estimation in Agile Software Development (ASD). Sci Comput Program. 2024;236. https://doi.org/10.1016/j.scico.2024.103114.
4. Hamid M, et al. An Intelligent Recommender and Decision Support System (IRDSS) for Effective Management of Software Projects. IEEE Access. 2020;8. https://doi.org/10.1109/ACCESS.2020.3010968.
5. Hamid M, et al. An intelligent decision support system for effective handling of IT projects. Journal of Intelligent & Fuzzy Systems. 2020;38:2635–47. https://doi.org/10.3233/JIFS-179550.
6. Nisa MU, Saqlain M, Abid M, Awais M, Stević Ž. Analysis of software effort estimation by machine learning techniques. Ingénierie des systèmes d'information. 2023;28(6):1–12. https://doi.org/10.18280/isi.280602.
7. Lavingia K, Patel R, Patel V, Lavingia A. Software Effort Estimation using Machine Learning Algorithms. Scalable Computing: Pract Experience. 2024;25(2):1276–85. https://doi.org/10.12694/scpe.v25i2.2213.

8. Fernández-Diego M, González-Ladrón-de-Guevara F, Méndez ER, Abrahão S, Insfran E. An update on effort estimation in agile software development: a systematic literature review. IEEE Access. 2020;8:160187–204. https://doi.org/10.1109/ACCESS.2020.3021664.
9. Arora M, V. S. An efficient ANFIS-EEBAT approach to estimate effort in scrum projects. Sci Rep. 2022;12(1):7974. https://doi.org/10.1038/s41598-022-11565-2.
10. Butt SA, Khan A, Khan U, Butt FZ, Jamil A. A software-based cost estimation technique in scrum using a developer's expertise. Adv Eng Softw. 2022;171:103159. https://doi.org/10.1016/j.advengsoft.2022.103159.
11. Hamid M, Zeshan F, Ahmad A, Ahmad F, Hamza MA, Khan ZA, et al. An intelligent recommender and decision support system (IRDSS) for effective management of software projects. IEEE Access. 2020;8:1–13. https://doi.org/10.1109/ACCESS.2020.3010968.
12. Suvvari, Sunil & Sawalkar, Rohini & KARAD, DR. The Effect of Team Size and Dynamics on Agile Estimation: Agile estimation. Innovative Research Thoughts. 2023;9.178–187. https://doi.org/10.36676/irt.v9.i5.1478.
13. Tawosi V, Moussa R, Sarro F. Agile Effort Estimation: Have We Solved the Problem Yet? Insights From the Replication of the GPT2SP Study. IEEE Trans Softw Eng. 2022;48(7):2637–53.
14. Tawosi V, Moussa R, Sarro F. Agile effort estimation: have we solved the problem yet? Insights from the replication of the GPT2SP study. IEEE Trans Softw Eng. 2023;49(4):1957–74. https://doi.org/10.1109/TSE.2022.3197775.
15. Alsaadi B, et al. Ensemble effort estimation for novice agile teams. Inf Softw Technol. 2024;170:107447. https://doi.org/10.1016/j.infsof.2024.107447.
16. Poženel M, Fürst L, Vavpotič D, Hovelja T. Agile effort estimation: comparing the accuracy and efficiency of planning poker, bucket system, and affinity estimation methods. Int J Softw Eng Knowl Eng. 2023;33(11n12):1923–50. https://doi.org/10.1142/s021819402350064x.
17. Rao KE, Rao GA. An innovative approach for software effort estimation by combining ensemble learning with recursive feature elimination. Evol Intell. 2021;14(1):151–62. https://doi.org/10.1007/s12065-020-00360-5.
18. Kaushik A, Sheoran K, Kapur R, Bhutani N, Singh B, Sharma H. SENSE: Software effort estimation using novel stacking ensemble learning. Innov Syst Softw Eng. 2024. https://doi.org/10.1007/s11334-024-00581-2.
19. Kumar RA, Vidyulatha G. Extreme learning machine applied to software development effort estimation. Afr J Biol Sci. 2024;6(7):746–57. https://doi.org/10.48047/AFJBS.6.7.2024.746-757.
20. Shidiq MB, Gata W, Kurniawan S, Saputra DD, Panggabean S. Time effort prediction of agile software development using machine learning techniques. Inspiration: Jurnal Teknologi Informasi Dan Komunikasi2023;13(2):39–48
21. Poženel M, Fürst L, Vavpotič D, Hovelja T. Agile effort estimation: comparing the accuracy and efficiency of planning poker, bucket system, and affinity estimation methods. Int J Softw Eng Knowl Eng. 2023;33(2):1–17. https://doi.org/10.1142/S021819402350064X.
22. Alsaadi B, Saeedi K. Data-driven effort estimation techniques of agile user stories: a systematic literature review. Artif Intell Rev. 2022;55:3245–70. https://doi.org/10.1007/s10462-021-10132-x.
23. Cubillos J, Aponte J, Gomez D, Rojas E. Agile effort estimation in Colombia: an assessment and opportunities for improvement. Sci Comput Program. 2024;236:103115. https://doi.org/10.1016/j.scico.2024.103115.
24. Alatawi MN. Forecasting the Software Engineering Model's Effort Estimation Using Constructive Cost Estimation Models. Iran J Comput Sci. 2024. https://doi.org/10.1007/s42044-024-00194-9.
25. Karna H, Gotovac S, Vicković L. Data mining approach to effort modeling on agile software projects. Informatica. 2020;44(2):231–9. https://doi.org/10.31449/inf.v44i2.2759.
26. Tawosi V, Moussa R, Sarro F. On the relationship between story point and development effort in agile open-source software, in *Proc. 16th ACM/IEEE Int. Symp. Empir. Softw. Eng. Meas. (ESEM'22)*, New York, NY, USA, 2022, pp. 1–12. https://doi.org/10.1145/3544902.3544921
27. Atoum I, Otoom AA. Enhancing software effort estimation with pre-trained word embeddings: a small-dataset solution for accurate story point prediction. Electronics. 2024;13(23):4843. https://doi.org/10.3390/electronics13234843.
28. Ersoy E, Bagriyanik S, Sozer H. On the Accuracy of Effort Estimations based on COSMIC Functional Size Measurement: A Case Study, in Proceedings of the 18th ACM / IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM '24), Barcelona, Spain, 2024, pp. 528–537. https://doi.org/10.1145/3674805.3695402
29. Tran HQ. Software development effort estimation using a fuzzy logic-based system within the context of the scaled agile framework. IOSR Journal of Computer Engineering (IOSR-JCE). 2020;22(1):10–9. https://doi.org/10.9790/0661-2201021019.
30. Turic M, Celar S, Dragicevic S, Vickovic L. Advanced Bayesian network for task effort estimation in agile software development. Appl Sci. 2023;13(16):9465. https://doi.org/10.3390/app13169465.
31. Abusaeed S, Khan SUR, Mashkoor A. A fuzzy AHP-based approach for prioritization of cost overhead factors in agile software development. Appl Soft Comput. 2023;133:109977. https://doi.org/10.1016/j.asoc.2022.109977.
32. Chahar V, Bhatia PK. Selection of effective parameters for optimizing software testing effort estimation. Applied Data Science and Smart Systems. 2023;85:1–9.
33. Shidiq AMB, Gata W, Kurniawan S, Saputra DD, Panggabean S. Time effort prediction of agile software development using machine learning techniques. Jurnal Teknologi Informasi dan Komunikasi. 2023;13(2):39–48.
34. Sánchez ER, Santacruz EFV, Maceda HC. Effort and cost estimation using decision tree techniques and story points in agile software development. Mathematics. 2023;11(6):1477. https://doi.org/10.3390/math11061477.
35. Gultekin M, Kalipsiz O. Story point-based effort estimation model with machine learning techniques. Int J Softw Eng Knowl Eng. 2020;30(1):43–66. https://doi.org/10.1142/S0218826220500030.
36. Kumar RA. Extreme Learning Machine Applied to Software Development Effort Estimation. African Journal of Bio Sciences. 2024;6(7):746–57. https://doi.org/10.48047/AFJBS.6.7.2024.746-757.
37. Yousif JH, Saini DK. Fuzzy and mathematical effort estimation models for web applications development. Appl Comput J. 2021;1(1):10–24. https://doi.org/10.52098/acj.20216.
38. Khalida R, Kusuma TM, Ramdhania KF. Fuzzy use case points as a basis for effort estimation. PIKSEL : Penelitian Ilmu Komputer Sistem Embedded and Logic. 2023;11(1):181–96. https://doi.org/10.33558/piksel.v11i1.6941.
39. de Morais RA. Deep learning based models for software effort estimation using story points in agile environments, M.S. thesis, Dept. Math. Phys. Sci., Concordia Univ. of Edmonton, Edmonton, AB, Canada, 2022.
40. Eichenwald M, Karkowski M, Müller R. Generalizable AI pipeline for the cost estimation of manufacturing system structures in special machine engineering. Procedia CIRP. 2024;126:140–5. https://doi.org/10.1016/j.procir.2024.08.314.

41. Rasheed M, Fatima I, Fatima D, et al. Effort estimation in scrum using AI. Autom Softw Eng. 2026;33:60. https://doi.org/10.1007/s10515-026-00607-y.
42. Jadhav A, Shandilya SK. Comparative analysis of machine learning algorithms for software development effort estimation. J Eng Res. 2023;11:362–76.
43. vom Brocke JW, Hevner R, Maedche A. Special Issue Editorial – Accumulation and Evolution of Design Knowledge in Design Science Research: A Journey Through Time and Space. J Assoc Inf Syst. 2020;21(3):520–44. https://doi.org/10.17705/1jais.00611.
44. Cubillos JA, Gomez J, Rojas D, Rojas E. Agile effort estimation in Colombia: an assessment and opportunities for improvement. Sci Comput Program. 2024;236:103115. https://doi.org/10.1016/j.scico.2024.103115.
45. Samuels JI. One-Hot Encoding and Two-Hot Encoding: An Introduction, 2024. [Online]. Available: https://www.researchgate.net/publication/371158812
46. Ross BC. Mutual information between discrete and continuous data sets. PLoS One. 2014;9(2):e87357. https://doi.org/10.1371/journal.pone.0087357.
47. Géron A. Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems. 2nd ed. Sebastopol, CA, USA: O'Reilly Media, Year:; 2019.
48. Hodson TO. Root-Mean-Square Error (RMSE) or Mean Absolute Error (MAE): when to use them or not. Geosci Model Dev. 2022;15:5481–7. https://doi.org/10.5194/gmd-15-5481-2022.
49. Priya Varshini AGK, Anitha K. Predictive analytics approaches for software effort estimation: a review. Indian J Sci Technol. 2020;13:2094–103. https://doi.org/10.17485/IJST/v13i21.573v.
50. Lavingia K, Patel R, Patel V, Lavingia A. Software effort estimation using machine learning algorithms. Scalable Computing: Pract Experience. 2024;25(2):1276–85. https://doi.org/10.12694/scpe.v25i2.2213.
51. Chen H. Enhancing Software Effort Estimation through Reinforcement Learning-based Project Management-Oriented Feature Selection, arXiv, arXiv:2403.16749v1, March 25, 2024 2024. [Online]. Available: https://arxiv.org/abs/2403.16749
52. Meiliana G, Daniella N, Wijaya NGE, Putra, Efata R. Agile Software Development Effort Estimation based on Product Backlog Items, presented at the 8th International Conference on Computer Science and Computational IntelligenceICCSCI (2023), 2023.
53. Arora M, Verma S, Kavita M, Wozniak J, Shafi, Ijaz MF. An efficient ANFIS-EEBAT approach to estimate effort of Scrum projects. Sci Rep. 2022;12:7974. https://doi.org/10.1038/s41598-022-11565-2.

Publisher's note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.